ACCENTURE 2026–2027 HIRING · ASSESSMENT 3 · CODING ROUND

Coding Assessment DSA + SQL + Frontend Zero to Hero

> 3 questions · 60 minutes · 1 DSA + 1 SQL + 1 Frontend_

Everything you need for the coding round, from the first line of input handling to medium-level problems. It covers how the platform works, a timing plan, the 8 DSA patterns that come up most, 21 fully solved problems in Java and Python, the SQL joins and GROUP BY patterns that are asked, and front-end tasks in HTML, CSS, JavaScript, React and Vue. Every program in this guide was compiled and run against its test cases.

Java · Python · C++ · C#

Arrays & strings

Hashing & two pointers

Recursion & numbers

SQL joins

DOM · React · Vue

3	60 min	21	10
Questions (1 each)	Suggested 25 · 15 · 20	Solved DSA problems	SQL patterns with real output

Independent preparation guide · Not affiliated with or endorsed by Accenture

Edition: 2026–2027

INSIDE THIS GUIDE

Contents

00 The coding round: format, platform and scoring

01 The 60-minute game plan

A1 Input/output templates (Java, Python, C++, C#)

A2 Complexity: will my solution pass?

A3 Arrays (P1–P3)

A4 Strings (P4–P7)

A5 Hashing (P8–P10)

A6 Two pointers & sliding window (P11–P13)

A7 Sorting-based problems (P14–P15)

A8 Recursion & backtracking (P16–P17)

A9 Number logic (P18–P21)

A10 Language cheat sheet

B SQL: joins, grouping, subqueries, 10 patterns

C Frontend: DOM, CSS, React, Vue

D Practice list, 10-day plan, final checklist

KNOW THE ROUND

00 The Coding Round

Three questions in **60 minutes**: one **DSA coding** problem, one **SQL query** and one **front-end task**. You choose the order. Each answer is graded by **hidden test cases**, so partially correct code earns partial marks.

Question	Time plan	What it looks like
C1 · DSA	~25 min	LeetCode **Easy to Medium**: arrays, strings, hashing, two pointers, basic recursion, sorting-based problems, number logic. Languages: **Java, Python, C++, C#, .NET** (some candidates only saw Java, Python and C#).
C2 · SQL	~15 min	One query on 1–3 tables: SELECT/WHERE/ORDER BY/GROUP BY/HAVING, **joins (main focus)**, aggregates, DISTINCT, LIMIT, basic subqueries. Exact column aliases are required.
C3 · Frontend	~20 min	Complete or fix a given HTML/CSS/JS template (HackerRank React-certification style): DOM manipulation, events, form validation, CSS selectors and flexbox, toggling classes, React and Vue basics.

How the coding editor works

Pick a language → a **template** appears (often a function to complete, sometimes a full program). **Run** / **Compile & Run** uses the sample input; **Submit** runs all test cases, including hidden ones. You can usually submit more than once; the last submission counts.

Important for Java users

The template may contain a line like `throw new Exception(...)` or `throw new UnsupportedOperationException()`. **Comment it out or delete it** before running, or your code will fail even if the logic is correct. Also keep the class name the template uses (often `Main` or `Solution`).

Scoring mindset

Hidden tests check **edge cases**: empty input, one element, all equal values, negative numbers, very large values. A simple solution that handles every edge case beats a clever one that crashes on N = 1.

USE EVERY MINUTE WELL

01 The 60-Minute Game Plan

Minutes	What to do
0 – 3	Open all three questions and read them. **Start with the one you are strongest at** to bank marks early (many candidates do SQL first; it is the shortest).
3 – 18	SQL (~15 min): write the query, Run, check column names and order against the expected output, Submit.
18 – 38	Frontend (~20 min): do the objectives one by one, keep ids/classes/text exact, check the Result pane after each change.
38 – 58	DSA (~20–25 min): brute force first if needed, then optimise. Test the samples and your own edge cases, then Submit.
58 – 60	Make sure every question has a submitted answer. Partial credit is better than nothing.

Solving a DSA problem in 6 steps

1 Read twice → 2 Note input limits → 3 Work 1 example by hand → 4 Pick a pattern → 5 Code + run samples → 6 Test edge cases, submit

Pattern hints in the wording

"pair / two numbers add up to" → hashing or two pointers · "longest / shortest substring" → sliding window · "sorted array" → two pointers or binary search · "count / frequency / duplicate / first unique" → HashMap · "all combinations / subsets" → recursion · "overlapping intervals" → sort first.

Edge-case checklist

N = 0 or 1 · all elements equal · negative numbers and zero · duplicates · already sorted / reverse sorted · maximum N (time) · values near 10^9 (use **long**) · empty string or spaces only · upper/lower case.

When stuck

Write the brute force (nested loops) and submit it: it passes the small tests. Then optimise with a HashMap, sorting or two pointers. Print intermediate values to debug, and **remove the prints** before submitting.

Output format is part of the answer

Print exactly what is asked: no extra text like "Answer:", no trailing spaces where a single space is expected, and the exact case (YES vs Yes). When a function is required, **return** the value instead of printing it.

PART A · BACKEND DSA CODING (C1)

Data Structures & Algorithms

Input handling, complexity, then the eight patterns that cover almost every Easy–Medium question, each with solved, tested programs in Java and Python.

A

A1 Input / Output Templates

READ INPUT, PRINT OUTPUT

Some questions give a **function to complete** (the platform reads input for you). Others expect a **full program** that reads from standard input (the keyboard) and prints to standard output. Learn one reading template by heart in your language. Each one below reads `N`, then N numbers, then a word.

JAVA — BUFFEREDREADER (FAST)

```java
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());
// line 1: N
        StringTokenizer st = new StringTokenizer(br.readLine());
// line 2: N numbers
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] =
Integer.parseInt(st.nextToken());
        String word = br.readLine().trim();
// line 3: a string
        long sum = 0;
        for (int x : a) sum += x;
        System.out.println(sum + " " + word.length());
    }
}
```

PYTHON — READ EVERYTHING AT ONCE

```python
import sys
data = sys.stdin.read().split()      # all tokens at once (fast)
n = int(data[0])                     # line 1: N
a = list(map(int, data[1:1 + n]))    # line 2: N numbers
word = data[1 + n]                   # line 3: a string
print(sum(a), len(word))
```

C++ — CIN WITH FAST I/O

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;                          // line 1: N
    vector<long long> a(n);
    for (auto &x : a) cin >> x;        // line 2: N numbers
    string word;
    cin >> word;                       // line 3: a string
    long long sum = 0;
    for (auto x : a) sum += x;
    cout << sum << " " << word.size() << endl;
    return 0;
}
```

C# — CONSOLE.READLINE + SPLIT

```csharp
using System;
using System.Linq;

class Program {
    static void Main() {
        int n = int.Parse(Console.ReadLine().Trim());
// line 1: N
        long[] a = Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries)
                          .Select(long.Parse).ToArray();
// line 2: N numbers
        string word = Console.ReadLine().Trim();
// line 3: a string
        Console.WriteLine(a.Sum() + " " + word.Length);
    }
}
```

Sample input `5 / 3 1 4 1 5 / hello` → output **14 5** (sum of the numbers and the length of the word). The Java, Python and C++ templates were compiled and run on this input; the C# template follows the same steps but was not executed for this guide.

Function-style template (very common)

You only write the body, for example `public static int secondLargest(int[] arr)` or `def second_largest(arr):` . **Return** the answer; do not print it and do not read input yourself. Keep the given signature unchanged.

Reading tips

Java: `Scanner` is simple but slow for 10^5+ numbers, so use `BufferedReader` . Mixing `nextInt()` and `nextLine()` leaves a newline behind, so call an extra `nextLine()` . Python: `input().split()` per line, or `sys.stdin.read().split()` for everything at once.

A2 Complexity in Practice

WILL IT BE FAST ENOUGH?

Online judges allow roughly **10^8 simple operations per second** (Python is about 10× slower than Java/C++). Use the input limit to choose the algorithm before you code:

Max N	Needed complexity	Typical approach
N ≤ 10–20	$O(2^N)$, $O(N!)$	Recursion, generate all subsets/permutations
N ≤ 1,000	$O(N^2)$	Two nested loops, brute force over pairs

Max N	Needed complexity	Typical approach
$N \le 10^5 - 10^6$	O(N log N) or O(N)	Sorting, HashMap, two pointers, sliding window, prefix sums
$N \le 10^7$	O(N)	One pass, sieve
$N \le 10^{18}$	O(log N) or O(√N)	Math, binary representation, digit loops, GCD

Integer overflow
Java/C++/C# `int` holds about ±2.1 × 10^9. Sums of many values, products and factorials need `long` / `long long`. Python integers never overflow.

Hidden O(N) costs
String `+=` in a loop (use StringBuilder / join), `list.remove(x)`, `x in list` (use a set), and `s.substring` inside a loop all cost O(N) each time.

Space
A HashMap or extra array of N items is fine for N ≤ 10^6. Deep recursion (N > ~10^4) can overflow the stack, so prefer loops then.

A3 THE MOST COMMON TOPIC

Arrays

Core moves
Traverse once and keep running values (max, min, sum, count) · **prefix sums** pre[i] = pre[i-1] + a[i], so a range sum is pre[r] − pre[l−1] · reverse a range with two indices · in-place rewrite with a write pointer · **Kadane** for the best subarray.

Questions you will meet
Second largest / smallest · max subarray sum · rotate by K · move zeros · missing number (n(n+1)/2 − sum) · leaders in an array · equilibrium index (prefix sums) · stock buy/sell once (track the min price so far).

P1 Second Largest Distinct Element

Easy · Arrays

Given an array of N integers, print the second largest **distinct** value. If it does not exist (all values equal, or N = 1), print -1.

Input: Line 1: N. Line 2: N integers.

Sample input	Output
6 12 35 1 10 34 1	34
3 5 5 3	3

Approach

1. Keep two variables: **first** (largest) and **second**, both starting at the smallest possible value.
2. For each x: if x > first, the old first becomes second and x becomes first.
3. Else if x < first and x > second, x becomes second (x == first is skipped, so duplicates of the max do not count).
4. One pass, no sorting needed.

Time **O(N)** · Space **O(1)** · ✓ 5 tests passed (Java + Python)

Trap: Sorting and taking arr[n-2] is wrong when the maximum repeats: [5, 5, 3] must give 3, not 5.

JAVA

```java
import java.util.*;

public class Main {
    static long secondLargest(long[] a) {
        long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
        for (long x : a) {
            if (x > first) {
                second = first;
                first = x;
            } else if (x < first && x > second) {
                second = x;
            }
        }
        return second == Long.MIN_VALUE ? -1 : second;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long[] a = new long[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextLong();
        System.out.println(secondLargest(a));
    }
}
```

PYTHON

```python
def second_largest(a):
    first = second = None
    for x in a:
        if first is None or x > first:
            second, first = first, x
        elif x < first and (second is None or x > second):
            second = x
    return -1 if second is None else second

n = int(input())
a = list(map(int, input().split()))
print(second_largest(a))
```

P2 Maximum Subarray Sum (Kadane's Algorithm)

Medium Arrays

Given N integers (some may be negative), print the largest sum of a **contiguous** subarray with at least one element.

Input: Line 1: N. Line 2: N integers.

Sample input	Output
9 -2 1 -3 4 -1 2 1 -5 4	6
3 -5 -2 -8	-2

Approach

1. **cur** = best sum of a subarray that ends at the current index; **best** = best seen so far.
2. For each x: cur = max(x, cur + x) — either start fresh at x or extend the previous subarray.
3. best = max(best, cur). Start both with the first element so an all-negative array works.

Time **O(N)** Space **O(1)** ✓ 4 tests passed (Java + Python)

Trap: Starting best at 0 prints 0 for an all-negative array; the answer must be the largest single element.

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long cur = sc.nextLong(), best = cur;
        for (int i = 1; i < n; i++) {
            long x = sc.nextLong();
            cur = Math.max(x, cur + x);
            best = Math.max(best, cur);
        }
        System.out.println(best);
    }
}
```

PYTHON

```python
n = int(input())
a = list(map(int, input().split()))
cur = best = a[0]
for x in a[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
print(best)
```

P3 Rotate an Array to the Right by K

Medium Arrays

Rotate an array of N integers to the right by K steps (K can be larger than N) and print it.

Input: Line 1: N and K. Line 2: N integers.

Sample input	Output
5 2 1 2 3 4 5	4 5 1 2 3
3 7 10 20 30	30 10 20

Approach

1. Rotating N times gives the same array, so first do **k = k % n**.
2. Reversal trick: reverse the whole array, then reverse the first k elements, then reverse the rest.
3. Example [1 2 3 4 5], k=2: reverse all → 5 4 3 2 1 → reverse first 2 → 4 5 3 2 1 → reverse rest → 4 5 1 2 3.

Time **O(N)** Space **O(1) extra** ✓ 4 tests passed (Java + Python)

Trap: Forgetting k % n gives an index error (or many wasted rotations) when K > N.

JAVA

```java
import java.util.*;

public class Main {
    static void reverse(int[] a, int i, int j) {
        while (i < j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), k = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        k %= n;
        reverse(a, 0, n - 1);
        reverse(a, 0, k - 1);
        reverse(a, k, n - 1);
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append(a[i]).append(i < n
- 1 ? " " : "");
        System.out.println(sb);
    }
}
```

PYTHON

```python
n, k = map(int, input().split())
a = list(map(int, input().split()))
k %= n
a = a[-k:] + a[:-k] if k else a      # slicing does the rotation
print(*a)
```

A4 CHARACTERS AND WORDS

Strings

Core moves
Loop over characters · count with `int[26]` (`c - 'a'`) or a HashMap · two pointers from both ends (palindromes) · split into words · build output with **StringBuilder** (Java) or `"".join(list)` (Python). Java Strings are **immutable**; compare them with `.equals()`, never `==`.

Questions you will meet
Palindrome · anagram · first unique character · reverse words · compress (aaabb → a3b2) · count vowels/consonants · toggle case · remove duplicates · longest common prefix · check rotation (s2 inside s1 + s1).

P4 Valid Palindrome (Ignore Case and Symbols)

Easy · Strings

A sentence is a palindrome if, after keeping only letters and digits and ignoring case, it reads the same forwards and backwards. Print **YES** or **NO**.

Input: One line of text (may contain spaces and punctuation).

Sample input	Output
`A man, a plan, a canal: Panama`	`YES`
`race a car`	`NO`

Approach

1. Two pointers: i from the left, j from the right.
2. Skip characters that are not letters or digits.
3. Compare lower-case characters; if they differ print NO. If the pointers meet, print YES.

Time **O(N)** · Space **O(1)** · ✓ 4 tests passed (Java + Python)

Trap: Comparing without skipping spaces/punctuation, or without ignoring case, fails "A man, a plan, a canal: Panama".

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.hasNextLine() ? sc.nextLine() : "";
        int i = 0, j = s.length() - 1;
        boolean ok = true;
        while (i < j) {
            char a = s.charAt(i), b = s.charAt(j);
            if (!Character.isLetterOrDigit(a)) { i++; continue;
}
            if (!Character.isLetterOrDigit(b)) { j--; continue;
}
            if (Character.toLowerCase(a) !=
Character.toLowerCase(b)) { ok = false; break; }
            i++; j--;
        }
        System.out.println(ok ? "YES" : "NO");
    }
}
```

PYTHON

```python
s = input()
clean = [c.lower() for c in s if c.isalnum()]
print("YES" if clean == clean[::-1] else "NO")
```

P5 Check if Two Strings are Anagrams

Easy · Strings

Two words are anagrams if they contain the same letters the same number of times (case-insensitive). Print **YES** or **NO**.

Input: Two lines, one word on each.

Sample input	Output
`Listen` `Silent`	`YES`
`rat` `car`	`NO`

Approach

1. If the lengths differ, the answer is NO.
2. Count letters with an int[26] array (or a HashMap/Counter): +1 for each letter of the first word, -1 for the second.
3. They are anagrams only if every count ends at 0.

Time **O(N)** · Space **O(1) (26 counters)** · ✓ 4 tests passed (Java + Python)

Trap: Sorting both strings also works (O(N log N)), but remember to lower-case them first.

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String a = sc.next().toLowerCase(), b =
sc.next().toLowerCase();
        if (a.length() != b.length()) {
System.out.println("NO"); return; }
        int[] cnt = new int[26];
        for (int i = 0; i < a.length(); i++) {
            cnt[a.charAt(i) - 'a']++;
            cnt[b.charAt(i) - 'a']--;
        }
```

PYTHON

```python
from collections import Counter

a = input().strip().lower()
b = input().strip().lower()
print("YES" if Counter(a) == Counter(b) else "NO")
```

```
        for (int c : cnt) if (c != 0) {
System.out.println("NO"); return; }
        System.out.println("YES");
    }
}
```

P6 Reverse the Words of a Sentence

Easy Strings

Reverse the order of the words in a sentence. Words are separated by one or more spaces; print them separated by a single space with no leading or trailing spaces.

Input: One line of text.

Sample input	Output
`the sky is blue`	`blue is sky the`
`  hello  world`	`world hello`

Approach

1. Split on whitespace: Java `trim().split("\\s+")`, Python `split()` (no argument) — both drop empty pieces.
2. Join the words in reverse order with a single space.

Time **O(N)** Space **O(N)** ✓ 3 tests passed (Java + Python)

Trap: split(" ") keeps empty strings when there are double spaces, giving extra spaces in the output.

JAVA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String line = sc.hasNextLine() ? sc.nextLine().trim() :
"";
        if (line.isEmpty()) { System.out.println(); return; }
        String[] w = line.split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = w.length - 1; i >= 0; i--) {
            sb.append(w[i]);
            if (i > 0) sb.append(' ');
        }
        System.out.println(sb);
    }
}
```

PYTHON

```
words = input().split()
print(" ".join(reversed(words)))
```

P7 String Compression (Run-Length Encoding)

Medium Strings

Compress a string by replacing each run of the same character with the character followed by the run length, e.g. **aaabbc → a3b2c1**.

Input: One line: a non-empty string of letters.

Sample input	Output
`aaabbc`	`a3b2c1`
`abc`	`a1b1c1`

Approach

1. Walk the string with index i; count how far the same character continues (index j).
2. Append the character and the count (j - i), then jump i to j.
3. Use a StringBuilder in Java (string + in a loop is slow, $O(N^2)$).

Time **O(N)** Space **O(N)** ✓ 4 tests passed (Java + Python)

Trap: Forgetting to output the last run after the loop ends (when using a single loop with a counter).

JAVA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        StringBuilder sb = new StringBuilder();
        int i = 0;
        while (i < s.length()) {
            int j = i;
            while (j < s.length() && s.charAt(j) == s.charAt(i))
j++;
            sb.append(s.charAt(i)).append(j - i);
            i = j;
        }
        System.out.println(sb);
    }
}
```

PYTHON

```
from itertools import groupby

s = input().strip()
print("".join(ch + str(len(list(run))) for ch, run in
groupby(s)))
```

A5

O(1) LOOKUPS

Hashing

The idea

A **HashMap** (dict) maps a key to a value and a **HashSet** (set) stores unique keys. Insert, lookup and delete take **O(1) on average**. Turn "have I seen this before?" from an O(N) search into an O(1) check. That usually turns O(N²) into O(N).

Typical uses

Frequency counting · duplicates (set size vs list size) · Two Sum (complement lookup) · first non-repeating · group anagrams (key = sorted word) · longest consecutive sequence · subarray sum = K (map of prefix sums).

P8 First Non-Repeating Character

Easy · Hashing

Print the first character of the string that appears exactly once. If there is none, print -1.

Input: One line: a string of lowercase letters.

Sample input	Output
leetcode	l
loveleetcode	v

Approach

1. Pass 1: count every character (int[26] or HashMap).
2. Pass 2: walk the string again in order and print the first character whose count is 1.
3. Two passes over the string keep the original order.

Time **O(N)** · Space **O(1)** · ✓ 4 tests passed (Java + Python)

Trap: Iterating over a HashMap instead of the string loses the original order (HashMap is unordered).

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) cnt[c - 'a']++;
        for (char c : s.toCharArray()) {
            if (cnt[c - 'a'] == 1) { System.out.println(c);
return; }
        }
        System.out.println(-1);
    }
}
```

PYTHON

```python
from collections import Counter

s = input().strip()
cnt = Counter(s)
print(next((c for c in s if cnt[c] == 1), -1))
```

P9 Two Sum (Return the Indices)

Easy · Hashing

Given N integers and a target T, print the 0-based indices **i < j** of the two numbers that add up to T (exactly one answer exists).

Input: Line 1: N and T. Line 2: N integers.

Sample input	Output
4 9 2 7 11 15	0 1
3 6 3 2 4	1 2

Approach

1. Brute force checks every pair: O(N²).
2. Better: a HashMap from value → index of the numbers seen so far.
3. For each a[j], if (T - a[j]) is already in the map, the answer is (map[T - a[j]], j). Otherwise store a[j].

Time **O(N)** · Space **O(N)** · ✓ 4 tests passed (Java + Python)

Trap: Storing all numbers first and then searching can pair an element with itself (e.g. T = 6 and a 3).

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long t = sc.nextLong();
        Map<Long, Integer> seen = new HashMap<>();
        for (int j = 0; j < n; j++) {
            long x = sc.nextLong();
            if (seen.containsKey(t - x)) {
                System.out.println(seen.get(t - x) + " " + j);
                return;
            }
            seen.put(x, j);
        }
    }
}
```

PYTHON

```python
n, t = map(int, input().split())
a = list(map(int, input().split()))
seen = {}
for j, x in enumerate(a):
    if t - x in seen:
        print(seen[t - x], j)
        break
    seen[x] = j
```

P10 Most Frequent Element (Smallest on a Tie)

Medium · Hashing

Print the value that appears most often. If several values share the highest frequency, print the smallest of them, followed by its frequency.

Input: Line 1: N. Line 2: N integers.

Sample input	Output
7 4 1 2 2 4 4 1	4 3
6 5 3 5 3 9 9	3 2

Approach

1. Count frequencies in a HashMap (Python: Counter).
2. Pick the entry with the highest count; on equal counts prefer the smaller value.
3. Compare with a clear rule: (count bigger) OR (count equal AND value smaller).

Time **O(N)** · Space **O(N)** · ✓ 4 tests passed (Java + Python)

Trap: Returning the first value reached in the map on a tie — HashMap order is not sorted or insertion order.

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<Integer, Integer> freq = new HashMap<>();
        for (int i = 0; i < n; i++) freq.merge(sc.nextInt(), 1,
Integer::sum);
        int bestVal = 0, bestCnt = 0;
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            int v = e.getKey(), c = e.getValue();
            if (c > bestCnt || (c == bestCnt && v < bestVal)) {
                bestVal = v;
                bestCnt = c;
            }
        }
        System.out.println(bestVal + " " + bestCnt);
    }
}
```

PYTHON

```python
from collections import Counter

n = int(input())
cnt = Counter(map(int, input().split()))
val, c = min(cnt.items(), key=lambda kv: (-kv[1], kv[0]))
print(val, c)
```

TWO INDICES INSTEAD OF TWO LOOPS

A6 Two Pointers & Sliding Window

Two pointers

(a) **Opposite ends** on a **sorted** array: pair sum, palindrome check, container problems. (b) **Same direction** (read / write pointer): remove duplicates, move zeros, partition. Each pointer moves at most N times, so the total is O(N).

Sliding window

A window [left, right] over an array or string. Grow right; when the window breaks a rule (repeat character, sum too big), shrink from left. Used for longest/shortest substring or subarray, max sum of K consecutive elements, and at most K distinct characters.

P11 Move Zeros to the End

Easy · Two Pointers

Move all 0s to the end of the array while keeping the order of the non-zero elements. Print the array.

Input: Line 1: N. Line 2: N integers.

Sample input	Output
5 0 1 0 3 12	1 3 12 0 0
3 0 0 0	0 0 0

Approach

1. Pointer **w** (write position) starts at 0.
2. Scan with i: whenever a[i] != 0, copy it to a[w] and move w forward.
3. Finally fill positions w..n-1 with 0. Non-zero order is preserved because we copy left to right.

Time **O(N)** · Space **O(1)** · ✓ 3 tests passed (Java + Python)

Trap: Swapping zeros with the last element breaks the order of the non-zero values.

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int w = 0;
        for (int i = 0; i < n; i++)
            if (a[i] != 0) a[w++] = a[i];
        while (w < n) a[w++] = 0;
        StringBuilder sb = new StringBuilder();
```

PYTHON

```python
n = int(input())
a = list(map(int, input().split()))
w = 0
for x in a:
    if x != 0:
        a[w] = x
        w += 1
for i in range(w, n):
    a[i] = 0
print(*a)
```

```
        for (int i = 0; i < n; i++) sb.append(a[i]).append(i < n
- 1 ? " " : "");
        System.out.println(sb);
    }
}
```

P12 Longest Substring Without Repeating Characters

Medium · Sliding Window

Print the length of the longest substring (contiguous) that has no repeated character.

Input: One line: a string (may be empty).

Sample input	Output
abcabcbb	3
bbbbb	1

Approach

1. Sliding window [left, right]. Map each character to the last index where it was seen.
2. When s[right] was seen inside the window (last ≥ left), move left to last + 1.
3. Update last[s[right]] = right and best = max(best, right - left + 1).

Time **O(N)** · Space **O(alphabet)** · ✓ 5 tests passed (Java + Python)

Trap: Moving left backwards: always use left = max(left, last + 1) (or the check last ≥ left).

JAVA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.hasNextLine() ? sc.nextLine() : "";
        Map<Character, Integer> last = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (last.containsKey(c) && last.get(c) >= left) left
= last.get(c) + 1;
            last.put(c, right);
            best = Math.max(best, right - left + 1);
        }
        System.out.println(best);
    }
}
```

PYTHON

```
import sys

s = sys.stdin.readline().rstrip("\n")
last = {}
left = best = 0
for right, c in enumerate(s):
    if last.get(c, -1) >= left:
        left = last[c] + 1
    last[c] = right
    best = max(best, right - left + 1)
print(best)
```

P13 Pair with Target Sum in a Sorted Array

Easy · Two Pointers

The array is sorted in ascending order. Print the first pair (smallest left value) whose sum equals T, or **NONE** if no pair exists.

Input: Line 1: N and T. Line 2: N sorted integers.

Sample input	Output
6 10 1 2 4 6 8 9	1 9
4 100 1 2 3 4	NONE

Approach

1. i at the start, j at the end.
2. sum = a[i] + a[j]. If sum == T, print the pair. If sum < T, move i right (need bigger). If sum > T, move j left.
3. Works because the array is sorted — no extra memory needed.

Time **O(N)** · Space **O(1)** · ✓ 4 tests passed (Java + Python)

Trap: Using i ≤ j allows a number to pair with itself; use i < j.

JAVA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long t = sc.nextLong();
        long[] a = new long[n];
        for (int k = 0; k < n; k++) a[k] = sc.nextLong();
        int i = 0, j = n - 1;
        while (i < j) {
            long sum = a[i] + a[j];
            if (sum == t) { System.out.println(a[i] + " " +
a[j]); return; }
            if (sum < t) i++; else j--;
        }
        System.out.println("NONE");
    }
}
```

PYTHON

```
n, t = map(int, input().split())
a = list(map(int, input().split()))
i, j = 0, n - 1
ans = "NONE"
while i < j:
    s = a[i] + a[j]
    if s == t:
        ans = f"{a[i]} {a[j]}"
        break
    if s < t:
        i += 1
    else:
        j -= 1
print(ans)
```

ORDER MAKES IT EASY

A7 Sorting-Based Problems

Why sort?
After sorting, equal items are neighbours (duplicates, grouping), overlapping intervals are adjacent, and two pointers work. Built-in sort is O(N log N): Java `Arrays.sort` / `Collections.sort`, Python `sorted()` / `list.sort()`, C++ `sort()`, C# `Array.Sort`.

Custom order
Java: `list.sort((a, b) -> ...)` returns negative / zero / positive; use `Integer.compare(a, b)` instead of `a - b` for large values. Python: `sorted(x, key=lambda v: (-count[v], v))` sorts by count descending, then value ascending.

P14 Merge Overlapping Intervals

Medium · Sorting

Given N intervals [start, end], merge all overlapping intervals (touching ones such as [1,3] and [3,5] also merge) and print the result sorted by start, one interval per line.

Input: Line 1: N. Next N lines: start end.

Sample input	Output
4 1 3 2 6 8 10 15 18	1 6 8 10 15 18
2 1 4 4 5	1 5

Approach

1. Sort the intervals by start.
2. Keep a list of merged intervals. For each interval, if it starts at or before the end of the last merged one, extend that end to max(end, current end).
3. Otherwise it starts a new merged interval.

Time **O(N log N) for sorting** · Space **O(N)** · ✓ 4 tests passed (Java + Python)

Trap: Not sorting first, or replacing the end instead of taking the max ([1,10] and [2,3] must stay [1,10]).

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] iv = new int[n][2];
        for (int i = 0; i < n; i++) { iv[i][0] = sc.nextInt();
iv[i][1] = sc.nextInt(); }
        Arrays.sort(iv, (x, y) -> Integer.compare(x[0], y[0]));
        List<int[]> out = new ArrayList<>();
        for (int[] cur : iv) {
            if (!out.isEmpty() && cur[0] <= out.get(out.size() -
1)[1]) {
                int[] last = out.get(out.size() - 1);
                last[1] = Math.max(last[1], cur[1]);
            } else {
                out.add(new int[]{cur[0], cur[1]});
            }
        }
        for (int[] x : out) System.out.println(x[0] + " " +
x[1]);
    }
}
```

PYTHON

```python
n = int(input())
iv = sorted(tuple(map(int, input().split())) for _ in range(n))
out = []
for s, e in iv:
    if out and s <= out[-1][1]:
        out[-1][1] = max(out[-1][1], e)
    else:
        out.append([s, e])
for s, e in out:
    print(s, e)
```

P15 Sort Characters by Frequency

Medium · Sorting

Sort the characters of a string by frequency, highest first. Characters with the same frequency are ordered alphabetically. Print the resulting string.

Input: One line: a string of lowercase letters.

Sample input	Output
tree	eert
cccaaa	aaaccc

Approach

1. Count each character.
2. Sort the distinct characters with a **custom comparator**: higher count first, then alphabetical.
3. Append each character count times.

Time **O(N + K log K), K = distinct characters** · Space **O(N)**

✓ 4 tests passed (Java + Python)

Trap: A comparator must be consistent: compare counts first, and only when equal compare characters.

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
```

PYTHON

```python
from collections import Counter

s = input().strip()
cnt = Counter(s)
```

```java
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) cnt[c - 'a']++;
        List<Character> chars = new ArrayList<>();
        for (int i = 0; i < 26; i++) if (cnt[i] > 0)
chars.add((char) ('a' + i));
        chars.sort((x, y) -> cnt[y - 'a'] != cnt[x - 'a']
                ? cnt[y - 'a'] - cnt[x - 'a']
                : x - y);
        StringBuilder sb = new StringBuilder();
        for (char c : chars)
sb.append(String.valueOf(c).repeat(cnt[c - 'a']));
        System.out.println(sb);
    }
}
```

```python
order = sorted(cnt, key=lambda c: (-cnt[c], c))
print("".join(c * cnt[c] for c in order))
```

A FUNCTION THAT CALLS ITSELF

A8 Recursion & Backtracking

Two parts, always
(1) **Base case**: the smallest input, answered directly; without it the recursion never stops (StackOverflowError). (2) **Recursive case**: solve a smaller version and combine.

Memoization
If the same arguments repeat (Fibonacci, stairs, grid paths), store the results in an array or map. The time drops from exponential to O(number of different states).

Backtracking
Choose → recurse → **undo the choice**. Generates subsets, permutations and combinations: $O(2^N)$ or O(N!), so only for small N (≤ 10–20).

P16 Climbing Stairs (Recursion + Memoization)

Easy · Recursion

You can climb 1 or 2 steps at a time. Print the number of distinct ways to reach the top of N stairs (1 ≤ N ≤ 80).

Input: One integer N.

Sample input	Output
2	2
5	8

Approach

1. Recurrence: ways(n) = ways(n-1) + ways(n-2), with ways(1) = 1 and ways(2) = 2.
2. Plain recursion recomputes the same values again and again: $O(2^N)$.
3. **Memoization**: store each answer the first time it is computed, so every n is solved once: O(N).
4. Use long (64-bit): the answer for N = 80 is above the int range.

Time **O(N) with memo** · Space **O(N)** · ✓ 5 tests passed (Java + Python)

Trap: Without memoization N = 45 already takes seconds; with int instead of long, large N overflows.

JAVA

```java
import java.util.*;

public class Main {
    static long[] memo = new long[100];

    static long ways(int n) {
        if (n <= 2) return n;
        if (memo[n] != 0) return memo[n];
        return memo[n] = ways(n - 1) + ways(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(ways(sc.nextInt()));
    }
}
```

PYTHON

```python
from functools import lru_cache

@lru_cache(maxsize=None)
def ways(n):
    if n <= 2:
        return n
    return ways(n - 1) + ways(n - 2)

print(ways(int(input())))
```

P17 Generate All Subsets (Backtracking)

Medium · Recursion

Print every subset of the given distinct letters, one per line, in lexicographic order. Print the empty subset as **{}**.

Input: One line: distinct lowercase letters (1 to 10 of them).

Sample input	Output
abc	{} a ab abc ac b bc c
ba	{} a ab b

Approach

1. Sort the letters. Recursive function go(start, current): print current, then for each i ≥ start, add letter i and recurse with start = i + 1.
2. This visits subsets in lexicographic order: {}, a, ab, abc, ac, b, bc, c.
3. There are 2^N subsets, so the work is $O(N \cdot 2^N)$.

Time **O(N · 2^N)** · Space **O(N) recursion depth**

✓ 3 tests passed (Java + Python)

Trap: Not removing the last letter after the recursive call (backtracking step) when using a shared StringBuilder/list.

JAVA

```java
import java.util.*;

public class Main {
    static char[] s;
    static StringBuilder out = new StringBuilder();

    static void go(int start, StringBuilder cur) {
        out.append(cur.length() == 0 ? "{}" :
cur.toString()).append('\n');
        for (int i = start; i < s.length; i++) {
            cur.append(s[i]);
            go(i + 1, cur);
            cur.deleteCharAt(cur.length() - 1);   // backtrack
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        s = sc.next().toCharArray();
        Arrays.sort(s);
        go(0, new StringBuilder());
        System.out.print(out);
    }
}
```

PYTHON

```python
s = sorted(input().strip())
out = []

def go(start, cur):
    out.append("".join(cur) or "{}")
    for i in range(start, len(s)):
        cur.append(s[i])
        go(i + 1, cur)
        cur.pop()                       # backtrack

go(0, [])
print("\n".join(out))
```

A9 MATH-FLAVOURED PROBLEMS
Number Logic

Digit tricks

Last digit: `n % 10` · remove the last digit: `n / 10` (Python `//`) · digit count: loop or `len(str(n))` · reverse: `rev = rev * 10 + n % 10` · palindrome number: reversed == original · sum of digits, Armstrong, strong, perfect and Harshad numbers all use the same loop.

Number theory essentials

Prime check: divide only up to √n · many primes: sieve · GCD: Euclid · LCM = a / gcd × b · even/odd: `n % 2` or `n & 1` · power of two: `n > 0 && (n & (n − 1)) == 0` · fast power by squaring.

P18 Count Primes up to N (Sieve of Eratosthenes)

Medium · Number Logic

Print how many prime numbers are ≤ N ($0 \le N \le 10^7$).

Input: One integer N.

Sample input	Output
10	4
1	0

Approach

1. Checking every number by trial division is O(N√N) — too slow for $N = 10^7$.
2. Sieve: mark all numbers as prime, then for each prime p starting at 2, mark p·p, p·p + p, … as not prime.
3. Only p up to √N are needed. Count what stays marked.

Time **O(N log log N)** · Space **O(N)** · ✓ 5 tests passed (Java + Python)

Trap: 0 and 1 are not prime; start crossing out at p·p (smaller multiples were already crossed out).

JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

PYTHON

```python
n = int(input())
if n < 2:
    print(0)
else:
    is_prime = bytearray([1]) * (n + 1)
```

```
        int n = sc.nextInt();
        if (n < 2) { System.out.println(0); return; }
        boolean[] composite = new boolean[n + 1];
        int count = 0;
        for (int p = 2; p <= n; p++) {
            if (composite[p]) continue;
            count++;
            for (long m = (long) p * p; m <= n; m += p)
composite[(int) m] = true;
        }
        System.out.println(count);
    }
}
```

```
    is_prime[0] = is_prime[1] = 0
    p = 2
    while p * p <= n:
        if is_prime[p]:
            is_prime[p * p::p] = bytearray(len(range(p * p, n +
1, p)))
        p += 1
    print(sum(is_prime))
```

P19 Armstrong Numbers in a Range

Easy · Number Logic

An Armstrong number equals the sum of its digits each raised to the number of digits ($153 = 1^3 + 5^3 + 3^3$). Print all Armstrong numbers between L and R (inclusive) separated by spaces, or -1 if there are none.

Input: One line: L R.

Sample input	Output
100 500	153 370 371 407
1 9	1 2 3 4 5 6 7 8 9

Approach

1. For each number, count its digits d (convert to string, or divide by 10 repeatedly).
2. Add digit^d for every digit and compare with the number.
3. Collect the matches; print -1 if the list is empty.

Time **O((R - L) · digits)** · Space **O(1)** · ✓ 4 tests passed (Java + Python)

Trap: Using the fixed power 3 for every number: 9474 has 4 digits, so it needs power 4.

JAVA

```
import java.util.*;

public class Main {
    static boolean isArmstrong(int x) {
        int d = String.valueOf(x).length();
        long sum = 0;
        for (int t = x; t > 0; t /= 10) sum += (long) Math.pow(t
% 10, d);
        return sum == x;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int l = sc.nextInt(), r = sc.nextInt();
        StringBuilder sb = new StringBuilder();
        for (int x = Math.max(l, 1); x <= r; x++) {
            if (isArmstrong(x)) sb.append(sb.length() > 0 ? " "
: "").append(x);
        }
        System.out.println(sb.length() == 0 ? "-1" :
sb.toString());
    }
}
```

PYTHON

```
l, r = map(int, input().split())
res = [x for x in range(max(l, 1), r + 1)
       if x == sum(int(c) ** len(str(x)) for c in str(x))]
print(*res if res else [-1])
```

P20 GCD and LCM of an Array

Easy · Number Logic

Print the GCD and the LCM of N positive integers. The LCM fits in a 64-bit integer.

Input: Line 1: N. Line 2: N positive integers.

Sample input	Output
3 12 18 24	6 72
2 7 13	1 91

Approach

1. Euclid: gcd(a, b) = gcd(b, a % b), and gcd(a, 0) = a.
2. gcd of many numbers: fold left — g = gcd(g, x).
3. lcm(a, b) = a / gcd(a, b) * b. Divide **before** multiplying to avoid overflow.

Time **O(N log(max))** · Space **O(1)** · ✓ 4 tests passed (Java + Python)

Trap: a * b / gcd(a, b) can overflow even when the final LCM fits; compute a / gcd * b instead.

JAVA

```
import java.util.*;

public class Main {
    static long gcd(long a, long b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    public static void main(String[] args) {
```

PYTHON

```
from math import gcd
from functools import reduce

n = int(input())
a = list(map(int, input().split()))
g = reduce(gcd, a)
l = reduce(lambda x, y: x // gcd(x, y) * y, a)
print(g, l)
```

```
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long g = 0, l = 1;
        for (int i = 0; i < n; i++) {
            long x = sc.nextLong();
            g = gcd(g, x);
            l = l / gcd(l, x) * x;
        }
        System.out.println(g + " " + l);
    }
}
```

P21 Binary Form and Number of Set Bits

Easy · Number Logic

For a non-negative integer N, print its binary representation and the number of 1-bits.

Input: One integer N ($0 \le N \le 10^{18}$).

Sample input	Output
10	1010 2
0	0 0

Approach

1. Repeatedly take N % 2 (the last bit) and divide by 2; the bits come out in reverse order.
2. Count the 1-bits along the way (or use n & (n - 1), which removes the lowest 1-bit).
3. Library shortcuts: Java Long.toBinaryString / Long.bitCount; Python bin(n)[2:] / bin(n).count("1").

Time **O(log N)** · Space **O(log N)** · ✓ 4 tests passed (Java + Python)

Trap: N = 0 must print 0 (a loop that runs while n > 0 prints nothing).

JAVA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        if (n == 0) { System.out.println("0 0"); return; }
        StringBuilder bits = new StringBuilder();
        int ones = 0;
        while (n > 0) {
            long b = n % 2;
            bits.append(b);
            ones += (int) b;
            n /= 2;
        }
        System.out.println(bits.reverse() + " " + ones);
    }
}
```

PYTHON

```
n = int(input())
b = bin(n)[2:]
print(b, b.count("1"))
```

A10

SAME IDEA, FOUR SYNTAXES

Language Cheat Sheet

Task	Java	Python	C++	C#
Dynamic list	`List<Integer> a = new ArrayList<>(); a.add(5);`	`a = []; a.append(5)`	`vector<int> a; a.push_back(5);`	`var a = new List<int>(); a.Add(5);`
Hash map count	`m.merge(x, 1, Integer::sum);`	`m[x] = m.get(x, 0) + 1` or `Counter`	`unordered_map<int,int> m; m[x]++;`	`m[x] = m.GetValueOrDefault(x) + 1;`
Set	`Set<Integer> s = new HashSet<>();`	`s = set()`	`unordered_set<int> s;`	`var s = new HashSet<int>();`
Sort	`Arrays.sort(arr); list.sort(null);`	`a.sort()` / `sorted(a)`	`sort(a.begin(), a.end());`	`Array.Sort(arr); list.Sort();`
Sort descending	`list.sort(Collections.reverseOrder());`	`a.sort(reverse=True)`	`sort(a.rbegin(), a.rend());`	`list.Sort((x, y) => y.CompareTo(x));`
String length · char	`s.length()` · `s.charAt(i)`	`len(s)` · `s[i]`	`s.size()` · `s[i]`	`s.Length` · `s[i]`
Substring	`s.substring(a, b)` (b excluded)	`s[a:b]`	`s.substr(a, len)`	`s.Substring(a, len)`
Split words	`s.trim().split("\\s+")`	`s.split()`	`stringstream ss(s); ss >> w`	`s.Split(' ', RemoveEmptyEntries)`
Build string	`StringBuilder sb; sb.append(x);`	`"".join(parts)`	`string r; r += c;`	`var sb = new StringBuilder();`
Reverse	`new StringBuilder(s).reverse()`	`s[::-1]`	`reverse(s.begin(), s.end());`	`new string(s.Reverse().ToArray())`
Char tests	`Character.isDigit(c)`, `isLetter`	`c.isdigit()`, `c.isalpha()`	`isdigit(c)`, `isalpha(c)`	`char.IsDigit(c)`

Task	Java	Python	C++	C#
Int ⇄ String	`Integer.parseInt(s)` · `String.valueOf(n)`	`int(s)` · `str(n)`	`stoi(s)` · `to_string(n)`	`int.Parse(s)` · `n.ToString()`
Max / min value	`Integer.MAX_VALUE`, `Long.MIN_VALUE`	`float('inf')`	`INT_MAX`, `LLONG_MIN`	`int.MaxValue`
64-bit integer	`long`	(automatic)	`long long`	`long`

The same solution in C++

For C++ users: Two Sum (P9) and Kadane (P2) are written below with STL containers.

TWO SUM (P9) IN C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n; long long t;
    cin >> n >> t;
    unordered_map<long long, int> seen;          // value -> index
    for (int j = 0; j < n; j++) {
        long long x; cin >> x;
        auto it = seen.find(t - x);
        if (it != seen.end()) { cout << it->second << " " << j <<
endl; return 0; }
        seen[x] = j;
    }
    return 0;
}
```

Input `4 9 / 2 7 11 15` → **0 1** (compiled with g++ and run)

KADANE (P2) IN C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n; cin >> n;
    long long cur, best;
    cin >> cur; best = cur;
    for (int i = 1; i < n; i++) {
        long long x; cin >> x;
        cur = max(x, cur + x);
        best = max(best, cur);
    }
    cout << best << endl;
    return 0;
}
```

Input `9 / -2 1 -3 4 -1 2 1 -5 4` → **6** (compiled with g++ and run)

Choosing a language

Use the one you write fastest without looking things up. Python is shortest (Counter, slicing, sorted with key) but slower for very large inputs. Java and C# are verbose but safe. C++ is fastest. Practise in one language before the test, and know its input reading by heart.

PART B · DATABASE SQL (C2)

SQL for the Coding Round

B

One query, about 15 minutes. Joins are the main focus, together with GROUP BY/HAVING, aggregates, DISTINCT, LIMIT and simple subqueries. Triggers, window functions and other advanced SQL are not asked.

B1 READ IT LIKE THE DATABASE DOES

Query Skeleton & Execution Order

```
SELECT   columns, aggregates AS alias
FROM     table1 t1
JOIN     table2 t2 ON t1.key = t2.key
WHERE    row conditions
GROUP BY grouping columns
HAVING   group conditions
ORDER BY columns DESC
LIMIT    n;
```

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Consequences of the order

WHERE runs before grouping, so **COUNT() cannot go in WHERE**; filter groups with HAVING. SELECT aliases can be used in ORDER BY but not in WHERE.

Aggregates and NULL

COUNT(*) counts rows; COUNT(col), SUM, AVG, MIN and MAX **ignore NULLs**. Test with `IS NULL`, never `= NULL`.

Aliases

`COUNT(*) AS total`. An alias with a space needs quotes: `AS "First Name"`, or backticks in MySQL. The grader compares column names, so copy them exactly.

B2 THE MAIN FOCUS

Joins



INNER JOIN
Only rows that match in both tables



LEFT JOIN
All left rows, plus matches (NULL where there is no match)



RIGHT JOIN
All right rows, plus matches from the left



SELF JOIN
A table joined with itself (employee ↔ manager)

Every SQL example below runs on these two tables. Arjun has no department and Finance has no employees, so you can see what each join does with unmatched rows.

EMPLOYEES

emp_id	name	dept_id	salary	manager_id
101	Asha	1	90000	*NULL*
102	Ravi	1	72000	101
103	Meena	1	72000	101
104	John	2	48000	101
105	Priya	2	56000	104
106	Kiran	3	39000	101
107	Arjun	*NULL*	61000	102
108	Sara	2	48000	104

DEPARTMENTS

dept_id	dept_name
1	Engineering
2	Sales
3	HR
4	Finance

B3 LEARN THESE AND MOST SQL QUESTIONS ARE FAMILIAR

Ten Patterns with Real Output

S1 Filter and sort

Employees earning more than 50000, highest first; ties by name.

```
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC, name;
```

REAL OUTPUT

name	salary
Asha	90000
Meena	72000
Ravi	72000
Arjun	61000
Priya	56000

S2 INNER JOIN

Each employee with their department name. Arjun has no department, so INNER JOIN drops him.

```sql
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
ORDER BY e.name;
```

REAL OUTPUT

name	dept_name
Asha	Engineering
John	Sales
Kiran	HR
Meena	Engineering
Priya	Sales
Ravi	Engineering
Sara	Sales

S3 LEFT JOIN + COUNT(column)

Every department with its head-count, including departments with nobody (Finance shows 0).

```sql
SELECT d.dept_name, COUNT(e.emp_id) AS total
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY total DESC, d.dept_name;
```

REAL OUTPUT

dept_name	total
Engineering	3
Sales	3
HR	1
Finance	0

S4 GROUP BY + HAVING

Departments whose average salary is above 50000.

```sql
SELECT d.dept_name, ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 50000
ORDER BY avg_salary DESC;
```

REAL OUTPUT

dept_name	avg_salary
Engineering	78000.0
Sales	50666.67

S5 SELF JOIN

Each employee with their manager's name (LEFT JOIN keeps Asha, who has no manager).

```sql
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

REAL OUTPUT

employee	manager
Asha	*NULL*
Ravi	Asha
Meena	Asha
John	Asha
Priya	John
Kiran	Asha
Arjun	Ravi
Sara	John

S6 Second highest salary

Use a subquery so ties at the top are handled (two people earn 72000).

```sql
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

REAL OUTPUT

second_highest
72000

S7 Top earner per department

A correlated subquery keeps everyone who earns their department's maximum.

```
SELECT d.dept_name, e.name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.salary = (SELECT MAX(x.salary) FROM employees x WHERE x.dept_id
= e.dept_id)
ORDER BY d.dept_name;
```

REAL OUTPUT

dept_name	name	salary
Engineering	Asha	90000
HR	Kiran	39000
Sales	Priya	56000

S8 RIGHT JOIN / anti-join

Departments with no employees: RIGHT JOIN keeps every department, then keep the unmatched ones.

```
SELECT d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

REAL OUTPUT

dept_name
Finance

S9 DISTINCT + LIMIT

The three highest distinct salaries.

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

REAL OUTPUT

salary
90000
72000
61000

S10 Duplicates

Salaries shared by more than one employee.

```
SELECT salary, COUNT(*) AS people
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1
ORDER BY salary;
```

REAL OUTPUT

salary	people
48000	2
72000	2

Common mistakes that fail hidden tests

INNER JOIN where "all customers / all departments" are required (use LEFT JOIN) · COUNT(*) after a LEFT JOIN (counts 1 instead of 0, so use COUNT(child.id)) · forgetting DISTINCT when one row can match many · `>=` vs `>` (read "more than" and "at least" carefully) · wrong ORDER BY direction, or no tie-breaker · alias spelled differently from the question · LIKE 'A%' (starts with) vs '%A%' (contains).

PART C · FRONTEND (C3)

HTML, CSS, JavaScript, React & Vue

C

You get a template with HTML, CSS and JS tabs (or a React/Vue component) and a list of objectives. Hidden tests open your page, click, type and check the DOM and the computed CSS.

C1

WHAT THE GRADER CHECKS

How Front-end Tasks Are Graded

Exact selectors
Tests find elements by the **id, class and text given in the question**. `id="p"` vs `id="para"`, or "Account Alert" vs "Account alert", is a fail even if the page looks right.

Computed style
CSS is checked through the browser's computed values: `#71E5F4` becomes `rgb(113, 229, 244)`, and a width becomes "120px". Any valid CSS that produces the value passes.

Behaviour
Tests click buttons and type into inputs, then check the result. "Remove" means `element.remove()`, not `display:none`. Read every constraint.

C2

SELECT → LISTEN → CHANGE

JavaScript DOM Essentials

Need	Code
Select	`document.getElementById("id")` · `document.querySelector("#id .cls")` (first) · `querySelectorAll(".item")` (all, use forEach)
Read / write text	`el.textContent = "Hi"` (safe) · `el.innerHTML = "<b>Hi</b>"` (parses HTML) · input: `input.value`
Classes	`el.classList.add / remove / toggle / contains("x")` · replace all: `el.className = "x"`
Styles & attributes	`el.style.display = "none"` · `el.setAttribute("data-name", "apple")` · `el.dataset.name` · `btn.disabled = true`
Create / remove	`const li = document.createElement("li"); list.appendChild(li);` · `el.remove()` · `parent.innerHTML = ""` (clear)
Events	`el.addEventListener("click" | "input" | "change" | "submit" | "keydown", e => {...})` · `e.preventDefault()` · `e.key === "Enter"` · `e.target`

TOGGLE A CLASS ON CLICK

```
// 1. select  2. listen  3. change the DOM
const btn = document.querySelector("#btn");
const p = document.getElementById("p");

btn.addEventListener("click", () => {
    p.className = "change";          // replace all classes
    // p.classList.add("change");    // add one class
    // p.classList.toggle("open");   // add if missing, remove if present
});
```

CREATE AND REMOVE ELEMENTS

```
function addTask() {
    const text = input.value.trim();
    if (!text) return;                          // ignore blank input
    const li = document.createElement("li");
    li.innerHTML = `<span></span><button class="delete">Delete</button>`;
    li.querySelector("span").textContent = text;   // textContent = safe text
    li.querySelector(".delete").addEventListener("click", () => {
        li.remove();                            // removes it from the page
        updateCount();
    });
    list.appendChild(li);
    input.value = "";
    updateCount();
}
input.addEventListener("keydown", e => { if (e.key === "Enter") addTask(); });
```

FORM VALIDATION

```
const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function validate() {
    const ok = pattern.test(email.value);
    emailError.textContent = email.value && !ok ? "Invalid email" : "";
    submitBtn.disabled = !(ok && password.value.length >= 8);
}
email.addEventListener("input", validate);
password.addEventListener("input", validate);

form.addEventListener("submit", e => {
    e.preventDefault();                     // stop the page reload
    success.textContent = "Registered!";
});
```

SEARCH FILTER WITH DATA-* AND DISPLAY

```
function filterProducts() {
    const term = search.value.toLowerCase();
    document.querySelectorAll(".product").forEach(item => {
        const name = item.dataset.name.toLowerCase();   // data-name attribute
        item.style.display = name.includes(term) ? "" : "none";
    });
}
search.value = "nana";
filterProducts();                           // run once on load
search.addEventListener("input", filterProducts);
```

C3

SELECTORS AND LAYOUT

CSS Essentials

Selector	Matches
`#id` · `.cls` · `p`	by id · by class · by tag
`.a.b`	an element with both classes
`.a .b` · `.a > .b`	descendant · direct child
`button:hover` · `li:first-child`	state · position
`[data-name="x"]`	by attribute

Which rule wins? Inline style > #id > .class / [attr] / :hover > tag. With equal weight, the **later** rule wins. `!important` overrides everything (avoid it).

Box model content → padding → border → margin. `box-sizing: border-box` makes width include padding and border. `display: none` removes the element from the layout; `visibility: hidden` keeps its space.

FLEXBOX, MULTI-CLASS, MEDIA QUERY

```
.container {
    display: flex;                    /* children in a row */
    justify-content: space-between;   /* main axis spacing */
    align-items: center;              /* cross axis alignment */
    gap: 20px;
}
.card { border-radius: 8px; padding: 16px; box-shadow: 0 2px 8px rg
.card h2 { color: #333333; font-size: 20px; }
.tab.active { background-color: #1d4ed8; color: white; }   /* two c
.hidden { display: none; }
@media (max-width: 600px) {
    .container { flex-direction: column; }                 /* stac
}
```

Flexbox in one look `display:flex` on the parent · `flex-direction` row/column · `justify-content` (main axis): flex-start, center, space-between, space-around · `align-items` (cross axis): stretch, center · `gap` · `flex:1` on children shares the space · `flex-wrap: wrap`.

C4

COMPONENTS AND STATE

React Essentials

Concept	Remember
Component	A function that returns JSX; its name starts with a capital letter. `export default App`.
JSX	`className` (not class), `{expression}`, one root element (or `<>...</>`), self-close `<input />`.
State	`const [count, setCount] = useState(0)`. Never change state directly; call the setter. Use `setCount(c => c + 1)` when the new value depends on the old one.
Events	`onClick={() => ...}`, `onChange={e => setX(e.target.value)}`. Pass a function; do not call it (`onClick={f}`, not `onClick={f()}`).
Controlled input	`value={x}` + `onChange` keeps the input and the state in sync.
Lists	`items.map(i => <li key={i.id}>{i.name}</li>)`. Every item needs a unique **key**.
Conditional	`{ok ? <A/> : <B/>}` · `{show && <A/>}`
Props	`<Card title="Hi" />` → `function Card({ title })`. Data flows down; callbacks flow up.
useEffect	`useEffect(() => {...}, [dep])` runs after render when dep changes; `[]` means once (fetching data, timers).

SEARCH LIST: STATE, DERIVED VALUE, MAP + KEY, CONDITIONAL

```
import React, { useState } from "react";

const products = ["Laptop", "Mouse", "Keyboard", "Monitor"];

function App() {
    const [query, setQuery] = useState("");                  // stat
    const visible = products.filter(p =>
        p.toLowerCase().includes(query.toLowerCase()));      // der

    return (
        <div>
            <input id="search" value={query}
                   onChange={e => setQuery(e.target.value)} />   {/
            <p id="total">Showing {visible.length} of {products.len
            {visible.length === 0
                ? <p id="empty">No products found</p>           // 
                : <ul>{visible.map(p =>
                      <li className="item" key={p}>{p}</li>)}</ul>}
        </div>
    );
}
export default App;
```

C5

TEMPLATES AND REACTIVITY

Vue 3 Essentials

Directive / option	Use
`{{ value }}`	Show data in the template (interpolation)
`v-model`	Two-way binding for input, textarea, select and checkbox
`@click` / `v-on:click`	Event handler (a method or a short expression)
`:attr` / `v-bind`	Bind an attribute: `:disabled="count >= 5"`, `:class="{ done: todo.done }"`
`v-for` + `:key`	Render a list: `v-for="t in todos" :key="t.id"`
`v-if` / `v-show`	v-if adds/removes the element from the DOM; v-show only toggles display

TEMPLATE

Directive / option	Use
`data()`	Returns the reactive state object (this.x in methods)
`computed`	Derived values that update automatically (remaining count, filtered list)
`methods`	Functions called from the template
`setup()` + `ref()`	Composition API: `const n = ref(0)` ; use `n.value` in JS, plain `n` in the template

```
<div id="app">
    <input id="newTodo" v-model="newTodo">
    <button id="add" @click="addTodo">Add</button>
    <ul>
        <li v-for="todo in todos" :key="todo.id"
            class="todo" :class="{ done: todo.done }">
            <input type="checkbox" v-model="todo.done">
            <span>{{ todo.text }}</span>
        </li>
    </ul>
    <p id="remaining">{{ remaining }} left</p>
    <p v-if="todos.length === 0">Nothing to do!</p>
</div>
```

APP CODE

```
const { createApp } = Vue;

createApp({
    data() {
        return { newTodo: "", nextId: 2,
                 todos: [{ id: 1, text: "Learn Vue", done: false }]
    },
    computed: {
        remaining() { return this.todos.filter(t => !t.done).length
    },
    methods: {
        addTodo() {
            const text = this.newTodo.trim();
            if (!text) return;
            this.todos.push({ id: this.nextId++, text, done: false
            this.newTodo = "";
        }
    }
}).mount("#app");
```

Front-end checklist before you submit

Every objective done? · ids, classes and texts copied exactly · colours in the right property (background-color vs color) · event attached to the right element · page loads with no red error in the Console · "remove" really removes · the initial state is correct on page load (filters and counters run once at start).

PART D · GET EXAM-READY

Practice List, Plan & Checklist

D

What to practise, in what order, and what to check on the day.

D1

AFTER THE SOLVED PROBLEMS

30 More Problems to Practise

Pattern	Problems (Easy → Medium)
Arrays	Missing number · leaders in an array · best time to buy and sell stock · equilibrium index (prefix sums) · product of array except self · maximum of every window of size K
Strings	Count vowels and consonants · longest common prefix · check if one string is a rotation of another · remove duplicate characters · largest word in a sentence · Roman to integer
Hashing	Contains duplicate · intersection of two arrays · group anagrams · longest consecutive sequence · subarray sum equals K · isomorphic strings
Two pointers / window	Remove duplicates from a sorted array · merge two sorted arrays · container with most water · 3-sum · max sum of K consecutive elements · smallest subarray with sum ≥ S
Sorting / recursion / numbers	Kth largest element · sort 0s, 1s and 2s · permutations of a string · power of a number (recursive) · palindrome number · sum of digits until one digit

D2

ONE TOPIC A DAY

10-Day Plan

Day	Focus (about 2 hours)
1	Choose your language. Learn the input template (A1) and the cheat sheet (A10). Solve P1–P3.

Day	Focus (about 2 hours)
2	Strings: P4–P7 + 3 problems from D1.
3	Hashing and two pointers: P8–P13.
4	Sorting and recursion: P14–P17. Write the recursion trees on paper.
5	Number logic: P18–P21 + digit problems from D1.
6	SQL: B1–B3. Retype all 10 patterns from memory, then do the SQL live practice (15 questions).
7	Frontend: C1–C3. HTML/CSS/JS questions in the front-end live practice platform.
8	React and Vue: C4–C5, then the React and Vue practice questions.
9	Full mock: 1 DSA + 1 SQL + 1 Frontend in 60 minutes with a timer. Review your mistakes.
10	Second full mock, then redo every problem you got wrong. Read the checklist below.

D3

READ THIS THE NIGHT BEFORE

Final Cheat Sheet

DSA

- Comment out `throw new Exception` in Java templates
- Return (function template) vs print (full program)
- Read the limits: 10^5 needs O(N log N) or better
- Pair/complement → HashMap · sorted → two pointers
- Substring/subarray "longest" → sliding window
- Frequency → int[26] or Counter
- Intervals → sort by start, then merge
- Recursion: base case first; memoize repeats
- Use long for sums and products
- Test N = 1, all equal, negatives, empty
- Exact output format, no debug prints

SQL

- Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
- Aggregate filter → HAVING, never WHERE
- "All X even without Y" → LEFT JOIN + COUNT(y.id)
- "No matching rows" → LEFT JOIN ... WHERE y.id IS NULL
- Manager/employee → self join with two aliases
- Second highest → MAX(...) WHERE < (SELECT MAX...)
- Top per group → correlated subquery with MAX
- Duplicates → GROUP BY ... HAVING COUNT(*) > 1
- Copy aliases and column order exactly
- DISTINCT when one row can match many

Frontend

- Copy ids, classes and texts exactly
- querySelector → addEventListener → change the DOM
- classList.add / remove / toggle; className replaces all
- textContent for text; value for inputs
- Forms: e.preventDefault() on submit
- Remove means el.remove()
- Run filters/counters once on load too
- React: useState + setter, className, key in map
- Vue: v-model, v-for + :key, v-if, computed, :class
- No red errors in the Console

Practise on the live platforms

This series also has two practice platforms. The **SQL Assessment Live Practice** has 15 questions with a 15-minute timer and 5 test cases each. The **Front-end Technology Live Practice** has 15 questions across HTML/CSS/JS, React and Vue, with a 20-minute timer and automatic tests. Use them for days 6–10.

Disclaimer: This is an independent preparation guide for informational purposes only, not affiliated with or endorsed by Accenture. The exam pattern is based on publicly reported candidate experiences and may change. All problems are original practice problems. Every Java and Python program was compiled and run (JDK 21, Python 3) against all of its listed test cases. The C++ programs were compiled with g++ and run. The C# template was not executed. SQL outputs come from SQLite; MySQL gives the same results for these queries.